Highlights

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl,Matthias Güdemann

- A machine-checked formalization of Crole's theorem that five definitions of alpha equivalence coincide.
- An isomorphism, formalized in Lean 4, between a de Bruijn representation and the quotient of named terms by alpha equivalence, following Norrish and Vestergaard's proof pearl.
- A roadmap toward a formalized foundation for untyped lambda calculus research, including further binding representations and the remaining formalizations in Norrish and Vestergaard's network of directly proved relationships.

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl[a,*], Matthias Güdemann[a,*]

[a] *Munich University of Applied Sciences, Department of Computer Science and Mathematics, Lothstraße 34, Munich, D-80335, Bavaria, Germany*

ARTICLE INFO

Keywords:
α-equivalence
Atom
λ-expression
Permutation action
Renaming
Variable binding
de Bruijn indices
formal verification
proof assistant
Lean
2020 MSC: 03B70
68V15
68V20

ABSTRACT

Alpha equivalence of lambda terms with named variables is defined in the literature in several ways: by an axiom for renaming a bound variable, under one of two side conditions on the new variable, or by rules that swap the two bound variables with a third one. Crole proved in detail that five such definitions give the same relation, and observed that published proof sketches of this folklore fact are sometimes incorrect. We present a formalization of Crole's theorem in the Lean 4 proof assistant, developed for the library CSLib. To our knowledge the theorem had not been mechanized before. The formalization supplies the proofs that Crole omits or only sketches. As a second result we formalize, following Norrish and Vestergaard, an isomorphism between named terms modulo alpha equivalence and pure de Bruijn terms, with their respective notions of beta-reduction. It connects the two representations of lambda terms within one library: through it, the Church–Rosser theorem for de Bruijn terms becomes a theorem about named terms. The statements in this paper are linked to their Lean sources.

1. Introduction

Terms of the lambda calculus are identified up to the names of their bound variables. For terms with named variables this identification is a relation on terms, alpha equivalence, and the literature defines it in more than one way. The traditional definition takes the congruence generated by an axiom for renaming a bound variable, where the new variable is required not to occur in the body at all [3, Definition 2.1.11], or only not to occur free in it [9, Definition 1.17]. Gabbay and Pitts [8] instead give syntax-directed rules in which the two bound variables are swapped with a third. That these definitions describe the same relation is folklore. Crole [5] points out that the proofs sketched in the literature are sometimes incorrect, and proves in detail that five such relations coincide.

Another way to deal with the names of bound variables is to do without them. In de Bruijn's representation a bound variable is a numeral that counts the binders between its occurrence and the binder it refers to. That nameless terms are isomorphic to named terms modulo alpha equivalence is stated, for instance, in [3, Appendix C]. Norrish and Vestergaard [12] give a proof mechanized in HOL4, for pure de Bruijn terms with the β-reduction of Nipkow [11].

This paper presents formalizations of both results in the Lean 4 proof assistant. They were developed for CSLib [4], a library of computer science for Lean 4 built on Mathlib, and extend its named representation of the untyped lambda calculus, which Section 2 recalls.

1. Section 3 formalizes Crole's theorem. Five relations on named terms, three defined by swapping and two by a renaming axiom, are equal (Theorem 3.9), whereas two further variants of the side condition define different relations (Proposition 3.10). To our knowledge the theorem had not been mechanized before. The formalization follows Crole's proof and supplies the arguments that [5] omits or only sketches (Table 1).
2. Section 4 formalizes the isomorphism of Norrish and Vestergaard between named terms modulo alpha equivalence and de Bruijn terms, with their respective notions of β-reduction (Theorem 4.13). The

*Corresponding author
chris.froeschl@hm.edu (C.A. Fröschl); matthias.guedemann@hm.edu (M. Güdemann)
ORCID(s): 0009-0006-3562-5395 (C.A. Fröschl); 0000-0002-1002-6023 (M. Güdemann)

quotient is taken by the alpha equivalence of CSLib, which is the first of Crole's five relations. As an application, the Church–Rosser theorem for de Bruijn terms yields the Church–Rosser theorem for named terms modulo alpha equivalence (Corollary 4.15). This result is not new as a mechanization. Its role here is to connect the two representations of lambda terms within one library.

The statements of Sections 2 to 4 carry a raised LEAN that links to the corresponding declaration in the Lean sources. A blueprint[1] presents both formalizations together, with a dependency graph[2] that shows how they are connected.

Both results are meant as parts of a library on which further work on the untyped lambda calculus can build. Section 5 discusses related work, Section 6 describes the directions we have in mind, in particular the remaining formalizations of the network of Norrish and Vestergaard [12, Figure 1], and Section 7 concludes.

2. CSLib and its named representation of the lambda calculus

2.1. CSLib

CSLib is a community-maintained library of foundational computer science formalizations for the Lean 4 theorem prover, built on top of Mathlib. [4] Its goal is to provide idiomatic, well-reviewed, and actively maintained formalizations of standard results, so that later research projects can build directly on a shared, trusted base rather than re-deriving basic definitions and lemmas from scratch. The work reported in this paper concerns CSLib's treatment of the untyped lambda calculus, and in particular its representation(s) of variable binding.

2.2. Named terms

CSLib currently represents untyped lambda terms in a standard named style: variables are drawn from an infinite set of atoms, and terms are built from variables, application, and abstraction. Our notation follows Crole [5], except that we write the concrete syntax of the lambda calculus where Crole writes $P(E_1, E_2)$ and $B([a]E)$ for a generic binary constructor and a generic binder; Crole notes that his proofs are insensitive to this choice [5, Section 2].

Definition 2.1 (Named lambda terms)**.** LEAN Let $\mathbb{A}$ be a set of atoms. The set Λ of named lambda terms is generated by

$$M \ ::= \ a \mid M\,M \mid \lambda a.\, M \qquad (a \in \mathbb{A}).$$

We write $\mathrm{occ}(M)$ LEAN for the finite set of all atoms occurring in M, including binding occurrences, and $\mathrm{free}(M)$ LEAN for the finite set of atoms occurring free in M, and abbreviate

$$a \not\triangleright M \iff a \notin \mathrm{occ}(M) \qquad \text{and} \qquad a \# M \iff a \notin \mathrm{free}(M).$$

In the second case we say that a is *fresh* for M. Both notations extend to lists, so that $z \not\triangleright a, b, M, M'$ means that z differs from a and b and occurs in neither M nor M'.

Crole assumes $\mathbb{A}$ to be countable, with equality of atoms as its only operation. In CSLib, $\mathbb{A}$ is an arbitrary type with decidable equality that comes with a function fresh from finite sets of atoms to atoms such that $\mathrm{fresh}(S) \notin S$; in particular $\mathbb{A}$ is infinite, and an atom avoiding any given finite set can be computed.

Three operations act on the atoms of a term. The first is purely textual and is the one CSLib's baseline alpha equivalence is phrased with.

Definition 2.2 (Renaming)**.** LEAN $M\langle a \mapsto b\rangle$ is M with every occurrence of a, free, bound or binding, replaced by b.

The second applies a bijection on atoms to every atom of a term, binding occurrences included.

[1] https://chris-anto-froeschl.github.io/untyped-lambda-calculus/
[2] https://chris-anto-froeschl.github.io/untyped-lambda-calculus/blueprint/dep_graph_document.html

Definition 2.3 (Permutation action [5, Section 2]). For a permutation π of $\mathbb{A}$, the term $\pi \cdot M$ is defined by

$$\pi \cdot a = \pi(a) \qquad \pi \cdot (M_1\, M_2) = (\pi \cdot M_1)\,(\pi \cdot M_2) \qquad \pi \cdot (\lambda a.\, M) = \lambda \pi(a).\, \pi \cdot M.$$

For the transposition $(a\ b)$ we call $(a\ b) \cdot M$ the *swap* of a and b in M. The *agreement set* of two permutations is $\mathrm{AS}(\pi, \pi') = \{\, a \in \mathbb{A} \mid \pi(a) = \pi'(a) \,\}$.

Unlike renaming (Definition 2.2), a swap is a bijection on terms and never identifies two atoms. The two operations agree when the new atom is unused: if $z \notin M$ then $(z\ a) \cdot M = M\langle a \mapsto z \rangle$.

The third is capture-avoiding substitution, which Crole writes $E\{a'/a\}$ for the substitution of an atom and which we write $M[a := N]$ for the substitution of an arbitrary term.

Definition 2.4 (Capture-avoiding substitution).

$$b[a := N] = \begin{cases} N & \text{if } b = a \\ b & \text{otherwise} \end{cases}$$

$$(M_1\, M_2)[a := N] = M_1[a := N]\ M_2[a := N]$$

$$(\lambda b.\, M)[a := N] = \begin{cases} \lambda b.\, M & \text{if } b = a \\ \lambda b.\, M[a := N] & \text{if } b \neq a \text{ and } b \# N \\ \lambda c.\, M\langle b \mapsto c \rangle[a := N] & \text{otherwise, where } c = \mathrm{fresh}(\mathrm{occ}(M) \cup \mathrm{occ}(N) \cup \{a, b\}) \end{cases}$$

For $M = \lambda a.\, a\, b$ with distinct atoms a and b the three operations differ: $M\langle a \mapsto b \rangle = \lambda b.\, b\, b$ identifies the two atoms, so that the free b is captured; $(a\ b) \cdot M = \lambda b.\, b\, a$ exchanges them; and $M[b := a] = \lambda c.\, c\, a$ for a fresh atom c, since substitution replaces only the free occurrence of b and renames the binder to avoid capturing a.

Definition 2.4 is, case for case, Gabbay's [6, Definition 5.13]: a function chooses the new atom fresh for a, b and the atoms of M and N, and the proof that the definition is well founded is his induction on the size of the term, which renaming preserves [6, Lemma 5.14]. Gabbay renames the binder with a substitution $M[b \mapsto c]$, which coincides with $M\langle b \mapsto c \rangle$ since c is fresh. Like Curry's substitution [3, Definition C.1], it is defined on all terms, but it renames b whenever b is free in N, whereas Curry keeps b also when a is not free in M. Renaming and substitution, a notion of alpha equivalence (Definition 3.1 below) and its basic theory, including the substitution lemma [3, 2.1.16], are due to Yin and Montesi[3] and are the starting point of our work: the alpha equivalence of CSLib is the reference point against which the other definitions are shown equivalent in Section 3.

3. Characterizations of alpha equivalence

The literature defines alpha equivalence of named terms in several ways: as the congruence generated by an axiom for renaming a bound atom, with one of two side conditions on the new atom, or, following Gabbay and Pitts [8], by a syntax-directed system of rules in which the two bound atoms are swapped with a third. That these definitions agree is folklore, but Crole [5] observes that the proof sketches in the literature are sometimes incorrect, and gives a detailed proof for five such relations. This section presents his result as it is formalized in CSLib. We keep Crole's names for the relations, the rules and the side conditions; each statement names its counterpart in [5], and a raised Lean links to its Lean source.

3.1. The five relations

The first three relations are syntax-directed and differ only in the side condition on the atom z that replaces the two bound atoms.

[3] https://github.com/leanprover/cslib/pull/458

Definition 3.1 ([5, Definitions 3.1–3.3]). The relation $\sim_{\mathrm{p}}$ on Λ is inductively defined by

$$\frac{}{a \sim_{\mathrm{p}} a}\ \textsc{atom} \qquad \frac{M_1 \sim_{\mathrm{p}} M_1' \quad M_2 \sim_{\mathrm{p}} M_2'}{M_1\, M_2 \sim_{\mathrm{p}} M_1'\, M_2'}\ \textsc{pcg} \qquad \frac{(z\ a) \cdot M \sim_{\mathrm{p}} (z\ b) \cdot M'}{\lambda a.\, M \sim_{\mathrm{p}} \lambda b.\, M'}\ \textsc{pi}\ [z \not\in a, b, M, M'].$$

The relations $\sim_{\mathrm{p}\#}$ and $\sim_{\mathrm{p}}^{1}$ are defined by the same rules, with the side condition of PI replaced by

$$[z \# a, b, M, M'] \qquad \text{and} \qquad [z \not\in M, M'],$$

respectively; we call the resulting rules PI# and PI1.

The relation $\sim_{\mathrm{p}}$ follows Gabbay and Pitts [8] and is CSLib's alpha equivalence $=_\alpha$. The remaining two relations are generated by the renaming axiom and are congruences by definition.

Definition 3.2 ([5, Definitions 3.4 and 3.5]). The relation $\sim_{\mathrm{r}}$ on Λ is inductively defined by

$$\frac{}{M \sim_{\mathrm{r}} M}\ \textsc{ref} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2}{M_2 \sim_{\mathrm{r}} M_1}\ \textsc{sym} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2 \quad M_2 \sim_{\mathrm{r}} M_3}{M_1 \sim_{\mathrm{r}} M_3}\ \textsc{trs}$$

$$\frac{M_1 \sim_{\mathrm{r}} M_1' \quad M_2 \sim_{\mathrm{r}} M_2'}{M_1\, M_2 \sim_{\mathrm{r}} M_1'\, M_2'}\ \textsc{pcg} \qquad \frac{M \sim_{\mathrm{r}} M'}{\lambda a.\, M \sim_{\mathrm{r}} \lambda a.\, M'}\ \textsc{bcg} \qquad \frac{}{\lambda a.\, M \sim_{\mathrm{r}} \lambda a'.\, M[a := a']}\ \alpha\ [a' \not\in a, M].$$

The relation $\sim_{\mathrm{r}\#}$ is defined by the same rules, with the side condition of α replaced by $[a' \# a, M]$; we call the resulting rule $\alpha\#$.

The relation $\sim_{\mathrm{r}}$ corresponds to the congruence $\equiv_\alpha$ of Barendregt [3, Definition 2.1.11], and $\sim_{\mathrm{r}\#}$ to that of Hindley and Seldin [9, Definition 1.17].

3.2. Properties of $\sim_{\mathrm{p}}$

Since $\sim_{\mathrm{p}}$ has no rules for reflexivity, symmetry and transitivity, these have to be proved. They were already available in CSLib, together with the two other basic facts below; the corresponding statements for $\equiv_\alpha$ are [9, Lemmas 1.19 and 1.21], proved in [9, Appendix A1].

Lemma 3.3. *The relation $\sim_{\mathrm{p}}$ is reflexive, symmetric and transitive. Moreover, if $M \sim_{\mathrm{p}} M'$ and $N \sim_{\mathrm{p}} N'$ then $\mathrm{free}(M) = \mathrm{free}(M')$ and $M[a := N] \sim_{\mathrm{p}} M'[a := N']$.*

The next two lemmas say that $\sim_{\mathrm{p}}$ is invariant under swapping, an instance of the equivariance principle of nominal techniques [6, Theorem 4.4], and that the action of a permutation on a term depends, up to $\sim_{\mathrm{p}}$, only on its values on the free atoms. Crole gives no source for either. That swapping preserves $\sim_{\mathrm{p}}$ is one of the two facts that Gabbay and Pitts call easily verified in their proof that $\sim_{\mathrm{p}}$ is alpha equivalence [8, proof of Proposition 2.2]; for the relation of nominal unification, Urban, Pitts and Gabbay prove it together with transitivity [15, (9) in the proof of Theorem 2.11]. The second part of Lemma 3.5 is their [15, Lemma 2.8], stated with the disagreement set, the complement of the agreement set, and with the same induction on the term. For a single swap it is also [7, Lemma 3.8]. The first part says that the atoms occurring in a term support it [8, Example 3.5(ii)].

Lemma 3.4 ([5, Lemma 6.1]). *If $M \sim_{\mathrm{p}} M'$ then $(u\ v) \cdot M \sim_{\mathrm{p}} (u\ v) \cdot M'$, for all atoms u, v.*

Lemma 3.5 ([5, Lemma 6.2]). *Let π, π' be permutations of $\mathbb{A}$ and M a term.*

1. *If $\mathrm{occ}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M = \pi' \cdot M$.*
2. *If $\mathrm{free}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M \sim_{\mathrm{p}} \pi' \cdot M$.*

The bridge between the two families of relations is that swapping with an unused atom is, up to alpha equivalence, substitution. For the renaming relations this is [5, Lemmas 6.3 and 6.4]. Crole calls Lemma 6.3 very similar to [6, Lemma 5.16] and points to [7] for a similar result. Gabbay proves it as a syntactic identity under the additional hypothesis that a is not bound in M [6, Lemma 5.16]; Gabbay and Mathijssen prove it for their alpha equivalence of ground terms, with z only fresh [7, Lemma 3.6(2)]. The version with $\sim_{\mathrm{p}}$ in place of $\sim_{\mathrm{r}}$ is the second fact used by Gabbay and Pitts [8, proof of Proposition 2.2]. Crole observes that their other direction needs Lemma 6.3 as well, and that this is missing from their sketch [5, p. 5].

Lemma 3.6. *If $z \not\triangleright M$ then $(z\ a) \cdot M \sim_{\mathrm{r}} M[a := z]$ and $(z\ a) \cdot M \sim_{\mathrm{r}\#} M[a := z]$.*

For $\sim_{\mathrm{p}}$ the corresponding statement compares two swaps.

Lemma 3.7 ([5, Lemma 6.5])**.** *Let $z_1, a_1, a_1', \ldots, z_n, a_n, a_n'$ be pairwise distinct atoms and let M be a term with $z_i \# M$ and $a_i' \# M$ for all i. Then*

$$(z_1\ a_1) \cdot \ldots \cdot (z_n\ a_n) \cdot M \ \sim_{\mathrm{p}}\ (z_1\ a_1') \cdot \ldots \cdot (z_n\ a_n') \cdot M[a_1 := a_1'] \ldots [a_n := a_n'].$$

Only the case $n = 1$ is used below. Crole omits the proof; ours does not follow the induction by which he handles the rule α in the proof of Theorem 4.4 [5, pp. 6–7]. For a single triple, $a' \# M$ gives $M[a := a'] \sim_{\mathrm{p}} (a\ a') \cdot M$: rename the bound occurrences of a' away (Lemma 3.8 below), use that substitution respects $\sim_{\mathrm{p}}$, and apply the $\sim_{\mathrm{p}}$-form of Lemma 3.6, which is the second fact of Gabbay and Pitts [8, proof of Proposition 2.2]. Then $(z\ a)$ and $(z\ a') \circ (a\ a')$ agree on $\mathrm{free}(M)$, and Lemma 3.5 applies. Renaming away first and transporting the result back along substitution is also how Hindley and Seldin reduce $\equiv_\alpha$ to its restricted steps [9, Lemmas A1.7 and A1.8], and how Crole proves Theorem 4.6 directly [5, p. 8]. The case of general n is an induction on n that commutes the remaining swaps past the first. Crole assumes $z_i \not\triangleright M$; freshness suffices, because a bound occurrence of z_i can be renamed away first:

Lemma 3.8 ([5, Lemma 6.6])**.** *If $a \# M$ then there is a term $\hat{M}$ with $a \not\triangleright \hat{M}$ and $\hat{M} \sim_{\mathrm{r}} M$, and likewise with $\sim_{\mathrm{p}}$ in place of $\sim_{\mathrm{r}}$.*

This lemma justifies Barendregt's variable convention [3, 2.1.13]. Our proof follows Crole's sketch [5, p. 16] except where the binder is a itself: renaming a to an unused z throughout the body removes every occurrence of a at once, and Lemma 3.6 relates the result to $M[a := z]$, so that no induction on the size of the term is needed. Crole calls the lemma well known; Hindley and Seldin show that any finitely many atoms can be made non-bound by renaming [9, Lemma A1.5(g)].

3.3. The characterization theorem

Theorem 3.9 (Crole [5, Theorems 4.1, 4.2, 4.4–4.6])**.** *For all terms $M, N \in \Lambda$ the following are equivalent:*

(i) $M \sim_{\mathrm{p}} N$;
(ii) $M \sim_{\mathrm{p}\#} N$;
(iii) $M \sim_{\mathrm{p}}^{1} N$;
(iv) $M \sim_{\mathrm{r}} N$;
(v) $M \sim_{\mathrm{r}\#} N$.

Proof. Each of (ii)–(v) is shown equivalent to (i), by showing that each of the two relations is closed under the rules of the other. Only the rules for abstractions need an argument.

(i) ⇔ (ii). Non-occurrence implies freshness, so PI is an instance of PI#. Conversely, let $(u\ a) \cdot M \sim_{\mathrm{p}} (u\ b) \cdot M'$ with $u \# a, b, M, M'$, and choose $z \not\triangleright u, a, b, M, M'$. By Lemma 3.4, $(z\ u) \cdot (u\ a) \cdot M \sim_{\mathrm{p}} (z\ u) \cdot (u\ b) \cdot M'$. The permutations $(z\ u) \circ (u\ a)$ and $(z\ a)$ agree on $\mathrm{free}(M)$, so the left-hand side is $\sim_{\mathrm{p}}$-related to $(z\ a) \cdot M$ by Lemma 3.5, and likewise for the right-hand side. Hence $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ b) \cdot M'$, and PI applies.

(i) ⇔ (iii). The rule PI1 is more general than PI only in allowing $z \in \{a, b\}$. If $z = a$, its premise reads $M \sim_{\mathrm{p}} (a\ b) \cdot M'$ with $a \not\triangleright M'$. For $z' \not\triangleright a, b, M, M'$, Lemma 3.4 and the first part of Lemma 3.5 give $(z'\ a) \cdot M \sim_{\mathrm{p}} (z'\ b) \cdot M'$. The case $z = b$ is symmetric.

(i) ⇔ (iv). For PI, let $(z\ a) \cdot M \sim_{\mathrm{r}} (z\ b) \cdot M'$ with $z \not\triangleright a, b, M, M'$. Lemma 3.6 gives $M[a := z] \sim_{\mathrm{r}} M'[b := z]$, and then

$$\lambda a.\, M \ \sim_{\mathrm{r}}\ \lambda z.\, M[a := z] \ \sim_{\mathrm{r}}\ \lambda z.\, M'[b := z] \ \sim_{\mathrm{r}}\ \lambda b.\, M'$$

by α, BCG and α. Conversely, REF, SYM and TRS hold for $\sim_{\mathrm{p}}$ by Lemma 3.3, and BCG by Lemma 3.4 and PI. For α, let $a' \not\triangleright a, M$ and choose $z \not\triangleright a, a', M, M[a := a']$. Then $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ a') \cdot M[a := a']$ by Lemma 3.7, which is the premise of PI for $\lambda a.\, M \sim_{\mathrm{p}} \lambda a'.\, M[a := a']$.

(i) ⇔ (v). As in the previous case: Lemma 3.6 covers $\sim_{\mathrm{r}\#}$, and Lemma 3.7 only requires $a' \# M$. □

Table 1
The lemmas of [5, Section 6.1] and the existence of substitution: their proofs in [5] and earlier sources.

In [5]	Proof there	Earlier source
Theorem A.1 (substitution)	deferred	[3, Definition C.1]; [6, Definition 5.13, Lemma 5.14]
Lemma 6.1	representative cases	[8, proof of Proposition 2.2]; [15, (9)]; [6, Theorem 4.4]
Lemma 6.2, part 1	omitted	[8, Example 3.5(ii)]
Lemma 6.2, part 2	given	[15, Lemma 2.8]; [7, Lemma 3.8]
Lemma 6.3	given	[6, Lemma 5.16]; [7, Lemma 3.6(2)]
Lemma 6.4	as for Lemma 6.3	as for Lemma 6.3
Lemma 6.5	details omitted	none for general n; for $n = 1$ our argument combines [8, proof of Proposition 2.2] and [9, Lemmas A1.7, A1.8]
Lemma 6.6	sketched, well known	[9, Lemma A1.5(g)]

In the last case Crole's argument differs from ours. He proves the rule α by an induction on M that fails for $\alpha\#$, where the substitution $M[a := a']$ may have to rename a binder, and introduces Lemma 3.7 for general n to repair it. Proving the case $n = 1$ under the freshness hypothesis serves both rules.

The side condition of PI cannot be weakened much further.

Proposition 3.10 ([5, Proposition 4.3]). *Let $\sim_{\mathrm{p}}^{2}$ and $\sim_{\mathrm{p}}^{3}$ be defined as $\sim_{\mathrm{p}}$, with the side condition of PI replaced by $[z \not\# a, b]$ and by no condition, respectively. Then $\sim_{\mathrm{p}}^{2} \neq \sim_{\mathrm{p}}$ and $\sim_{\mathrm{p}}^{3} \neq \sim_{\mathrm{p}}$.*

Proof. For distinct atoms a, b, z we have $\lambda a.\, a\, z \sim_{\mathrm{p}}^{2} \lambda b.\, b\, a$ and $\lambda a.\, b\, a \sim_{\mathrm{p}}^{3} \lambda b.\, a\, b$, but in both cases the two terms have different free atoms, so they are not related by $\sim_{\mathrm{p}}$ (Lemma 3.3). □

3.4. The formalization

The formalization follows [5] closely, and we found no errors in it. To our knowledge the result had not been mechanized before; Crole mentions an Isabelle/HOL development in progress [5, Section 7], of which we could find no published account. Our contribution is the four relations other than $\sim_{\mathrm{p}}$, Theorem 3.9, Proposition 3.10, and Lemmas 3.4–3.8. The definition of $\sim_{\mathrm{p}}$ and Lemma 3.3 belong to the named representation that this work builds on (Section 2.2). The first part of the development is under review for inclusion in CSLib[4]; the remainder is to follow in the same way.

What a formalization has to add to [5] are the proofs that the paper leaves to the reader or to the literature. Table 1 lists the lemmas of [5, Section 6.1] that the characterization needs, how much of their proofs Crole gives, and where the statement or its argument appears earlier in the literature. Lemma 6.7, which Crole uses only for a second, direct proof of Theorem 4.6, is not needed and not formalized. Crole names a source only for Lemma 6.3; the other entries in the last column are ours. None of the omitted proofs held a surprise.

Three statements differ from their counterparts in [5]. The rules PI and PI1 are stated in CSLib with renaming (Definition 2.2) in place of the swap, which is the same under their side conditions. Lemma 3.5 is proved for arbitrary bijections on $\mathbb{A}$, where Crole assumes finite support; the difference is immaterial for terms [6, Remark 2.25]. And Lemma 3.7 has the weaker hypothesis discussed above. Crole's second contribution, alpha equivalence of program contexts [5, Section 5], is not covered.

4. An isomorphism with de Bruijn terms

De Bruijn indices remove the need for alpha equivalence altogether by replacing bound variable names with numerals counting binder depth. Norrish and Vestergaard [12] give a direct, mechanized proof that the pure de Bruijn calculus is isomorphic to the alpha-quotiented named calculus, so that a result proved on de Bruijn terms is a result about lambda terms in the ordinary sense. We formalize this isomorphism for the named terms and the de Bruijn terms of CSLib. By Theorem 3.9 it does not matter which of the five alpha equivalences is quotiented by; the formalization uses $\sim_{\mathrm{p}}$.

[4] https://github.com/leanprover/cslib/pull/741

We keep the notation of [12] on the de Bruijn side, where t, u, s range over de Bruijn terms and i, j, k, n over indices. As in [12], M and N range over alpha-equivalence classes in this section.

4.1. De Bruijn terms

The de Bruijn terms, their substitution and β-reduction, and the proof of the Church–Rosser theorem used in Section 4.6 are not ours: they are due to Wang, the syntax and substitution jointly with Radziwill[5].

Definition 4.1 (De Bruijn terms [12, Section 2.2]). The set dB of de Bruijn terms is generated by

$$t \ ::= \ \mathsf{dV}\, i \mid \mathsf{dAPP}\, t\, t \mid \mathsf{dABS}\, t \qquad (i \in \mathbb{N}).$$

Lifting and substitution are defined as in Nipkow's formalization [11]:

$$\begin{aligned}
\mathsf{lift}\,(\mathsf{dV}\, i)\, k &= \text{if } i < k \text{ then } \mathsf{dV}\, i \text{ else } \mathsf{dV}\,(i+1)\\
\mathsf{lift}\,(\mathsf{dAPP}\, t\, u)\, k &= \mathsf{dAPP}\,(\mathsf{lift}\, t\, k)\,(\mathsf{lift}\, u\, k)\\
\mathsf{lift}\,(\mathsf{dABS}\, t)\, k &= \mathsf{dABS}\,(\mathsf{lift}\, t\,(k+1))\\
\\
\mathsf{nsub}\, s\, k\,(\mathsf{dV}\, i) &= \text{if } k < i \text{ then } \mathsf{dV}\,(i-1) \text{ else if } i = k \text{ then } s \text{ else } \mathsf{dV}\, i\\
\mathsf{nsub}\, s\, k\,(\mathsf{dAPP}\, t\, u) &= \mathsf{dAPP}\,(\mathsf{nsub}\, s\, k\, t)\,(\mathsf{nsub}\, s\, k\, u)\\
\mathsf{nsub}\, s\, k\,(\mathsf{dABS}\, t) &= \mathsf{dABS}\,(\mathsf{nsub}\,(\mathsf{lift}\, s\, 0)\,(k+1)\, t)
\end{aligned}$$

The function nsub substitutes for the index k and at the same time decrements all larger free indices, which is what β-contraction needs when a binder disappears.

Definition 4.2 ([12, Definition 1]). The relation $\to_{\mathrm{d}}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\,(\mathsf{dABS}\, t)\, u \to_{\mathrm{d}} \mathsf{nsub}\, u\, 0\, t} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dABS}\, t \to_{\mathrm{d}} \mathsf{dABS}\, u}$$

$$\frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\, t\, s \to_{\mathrm{d}} \mathsf{dAPP}\, u\, s} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\, s\, t \to_{\mathrm{d}} \mathsf{dAPP}\, s\, u}\,.$$

4.2. The alpha-quotiented calculus

Definition 4.3 ([12, Section 2.1]). Λ_α is the quotient of Λ by $\sim_{\mathrm{p}}$. We write $[M]_\alpha$ for the class of a term $M \in \Lambda$. Application, abstraction and substitution are lifted from Λ to Λ_α:

$$[M]_\alpha\,[N]_\alpha = [M\,N]_\alpha \qquad \lambda a.\,[M]_\alpha = [\lambda a.\,M]_\alpha \qquad [M]_\alpha[a := [N]_\alpha] = [M[a := N]]_\alpha.$$

These operations are well defined because $\sim_{\mathrm{p}}$ is a congruence and is preserved by substitution. Abstraction on Λ_α is not injective, so Λ_α is not a free algebra.

Definition 4.4 (β-reduction on Λ_α). The relation $\to_\beta$ on Λ_α is the least relation satisfying

$$\frac{}{(\lambda a.\,M)\,N \to_\beta M[a := N]} \qquad \frac{M \to_\beta N}{\lambda a.\,M \to_\beta \lambda a.\,N}$$

$$\frac{M \to_\beta N}{M\,Z \to_\beta N\,Z} \qquad \frac{M \to_\beta N}{Z\,M \to_\beta Z\,N}\,.$$

This is the system $\lambda^{\mathrm{N}}_\alpha$ of [12]: substitution and reduction are defined on the quotient itself and not on raw terms.

Two systems $\langle X, \to_X\rangle$ and $\langle Y, \to_Y\rangle$ are *isomorphic* if there is a bijection $f\colon X \to Y$ with $x_1 \to_X x_2 \iff f(x_1) \to_Y f(x_2)$ for all x_1, x_2 [12, Section 2.3].

[5] https://github.com/leanprover/cslib/pull/475

4.3. An intermediate formalism

The proof goes through a second presentation dB' of the same type dB that is built to resemble Λ_α. It replaces nsub by a substitution that leaves the other indices alone, and dABS by an abstraction over a chosen index.

Definition 4.5 ([12, Definitions 4 and 5]). LEMMA LEMMA

$$\begin{aligned}
\mathsf{sub}\, s\, k\, (\mathsf{dV}\, i) &= \text{if } i = k \text{ then } s \text{ else } \mathsf{dV}\, i \\
\mathsf{sub}\, s\, k\, (\mathsf{dAPP}\, t\, u) &= \mathsf{dAPP}\, (\mathsf{sub}\, s\, k\, t)\, (\mathsf{sub}\, s\, k\, u) \\
\mathsf{sub}\, s\, k\, (\mathsf{dABS}\, t) &= \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{lift}\, s\, 0)\, (k+1)\, t) \\
\mathsf{dLAM}\, i\, t &= \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{dV}\, 0)\, (i+1)\, (\mathsf{lift}\, t\, 0))
\end{aligned}$$

Like abstraction on Λ_α, dLAM is not injective. Every dABS term is a dLAM term over any index that is not free in it LEMMA [12, Lemma 6].

Definition 4.6 ([12, Definition 7]). LEMMA The relation $\to_{\mathrm{d}'}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\, (\mathsf{dLAM}\, i\, t)\, u \to_{\mathrm{d}'} \mathsf{sub}\, u\, i\, t} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dLAM}\, i\, t \to_{\mathrm{d}'} \mathsf{dLAM}\, i\, u}$$

$$\frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\, t\, s \to_{\mathrm{d}'} \mathsf{dAPP}\, u\, s} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\, s\, t \to_{\mathrm{d}'} \mathsf{dAPP}\, s\, u}\,.$$

4.4. The bijection

Fix a bijection between $\mathbb{A}$ and $\mathbb{N}$, written with a hat in both directions as in [12]: $\hat{a}$ is the number of the atom a and $\hat{\imath}$ the atom with number i.

Theorem 4.7 ([12, Theorem 11]). LEMMA *There is a bijection* $f\colon \Lambda_\alpha \to \mathrm{dB}$ *with*

$$f([a]_\alpha) = \mathsf{dV}\, \hat{a} \qquad f(M\, N) = \mathsf{dAPP}\, (f\, M)\, (f\, N) \qquad f(\lambda a.\, M) = \mathsf{dLAM}\, \hat{a}\, (f\, M). \tag{1}$$

Since Λ_α is not a free algebra, the equations (1) do not define a function by themselves. Norrish and Vestergaard obtain f from Pitts' recursion principle for Λ_α [13], for which dB has to be shown to be a nominal set. The formalization takes a more elementary route: the equations (1) define a function on raw terms by structural recursion LEMMA, this function is shown to respect $\sim_{\mathrm{p}}$ LEMMA, and f is its lift to the quotient LEMMA. Injectivity LEMMA is proved by structural induction on raw terms. Its abstraction case rests on the fact that dLAM is injective up to a transposition: $\mathsf{dLAM}\, i\, t = \mathsf{dLAM}\, j\, u$ holds exactly if $i = j$ and $t = u$, or $i \neq j$, the index i is not free in u, and t is u with i and j exchanged LEMMA. Surjectivity LEMMA is proved, as in [12], by complete induction on the size of the de Bruijn term, using Lemma 6 of [12] for dABS terms.

The bijection turns substitution on Λ_α into the index-preserving substitution sub, and with it β-reduction into $\to_{\mathrm{d}'}$.

Lemma 4.8 ([12, proof of Theorem 12]). LEMMA $f(M[a := N]) = \mathsf{sub}\, (f\, N)\, \hat{a}\, (f\, M)$.

Theorem 4.9 ([12, Theorem 12]). LEMMA *For all* $M, N \in \Lambda_\alpha$*:* $M \to_\beta N$ *if and only if* $f(M) \to_{\mathrm{d}'} f(N)$.

4.5. The isomorphism

It remains to show that $\to_{\mathrm{d}}$ and $\to_{\mathrm{d}'}$ are the same relation on dB. Their rules for redexes are related by an identity between sub and nsub LEMMA [12, Lemma 13]; the difficulty is to match the congruence rule for dABS with the one for dLAM. This is where permutations enter on the de Bruijn side. A permutation π of atoms acts on a de Bruijn term through the free indices LEMMA:

$$\pi \cdot (\mathsf{dV}\, i) = \mathsf{dV}\, \widehat{\pi(\hat{\imath})} \qquad \pi \cdot (\mathsf{dAPP}\, t\, u) = \mathsf{dAPP}\, (\pi \cdot t)\, (\pi \cdot u) \qquad \pi \cdot (\mathsf{dABS}\, t) = \mathsf{dABS}\, (\mathsf{inc}(\pi) \cdot t),$$

where $\mathsf{inc}(\pi)$ is π shifted by one index, so that it fixes the index bound by the abstraction [12, Definition 8]. A relation R on dB is *equivariant* if $R\, (\pi \cdot t)\, (\pi \cdot u) \iff R\, t\, u$ for all π.

Theorem 4.10 ([12, Theorem 10]). *For every permutation π of finite support:*

$$\pi \cdot (\mathsf{sub}\, t\, j\, u) = \mathsf{sub}\, (\pi \cdot t)\, \widehat{\pi(j)}\, (\pi \cdot u)$$

$$\pi \cdot (\mathsf{dLAM}\, i\, t) = \mathsf{dLAM}\, \widehat{\pi(i)}\, (\pi \cdot t)$$

$$\pi \cdot t \to_{\mathrm{d}'} \pi \cdot u \iff t \to_{\mathrm{d}'} u$$

The key observation of [12] is that lifting is itself the action of a permutation [12, Lemma 14], and so is the substitution of a fresh index. A dABS term and its presentation as a dLAM term therefore differ by a permutation, which an equivariant relation cannot see.

Theorem 4.11 ([12, Theorem 15]). *Let R be an equivariant relation on* dB. *Then*

$$\big(\forall t\, u.\ R\, t\, u \Rightarrow R\, (\mathsf{dABS}\, t)\, (\mathsf{dABS}\, u)\big) \iff \big(\forall i\, t\, u.\ R\, t\, u \Rightarrow R\, (\mathsf{dLAM}\, i\, t)\, (\mathsf{dLAM}\, i\, u)\big).$$

Theorem 4.11 applies to $\to_{\mathrm{d}'}$ by Theorem 4.10. To apply it to $\to_{\mathrm{d}}$ one needs equivariance of $\to_{\mathrm{d}}$, and hence the behaviour of nsub under a permutation [12, Lemma 17].

Theorem 4.12 ([12, Theorem 19]). *For all $t, u \in$ dB: $t \to_{\mathrm{d}'} u$ if and only if $t \to_{\mathrm{d}} u$.*

Theorem 4.13 ([12, Theorem 20]). *The systems $\langle \Lambda_\alpha, \to_\beta \rangle$ and $\langle \mathrm{dB}, \to_{\mathrm{d}} \rangle$ are isomorphic, as witnessed by the bijection f of Theorem 4.7: for all $M, N \in \Lambda_\alpha$,*

$$M \to_\beta N \iff f(M) \to_{\mathrm{d}} f(N).$$

Proof. From Theorems 4.9 and 4.12. □

The same method applies to η-reduction. Let $\to_\eta$ be the relation on Λ_α generated by the congruence rules of Definition 4.4 and the contraction of $\lambda a.\, M\, a$ to M for $a \# M$, and let $\to_{\mathrm{ne}}$ be Nipkow's η-reduction on dB, which contracts $\mathsf{dABS}\, (\mathsf{dAPP}\, t\, (\mathsf{dV}\, 0))$ to $\mathsf{nsub}\, u\, 0\, t$ when 0 is not free in t [12, Section 4.4].

Theorem 4.14 ([12, Section 4.4]). *For all $M, N \in \Lambda_\alpha$: $M \to_\eta N$ if and only if $f(M) \to_{\mathrm{ne}} f(N)$.*

4.6. Using the isomorphism

An isomorphism of reduction systems transports every property that is stated in terms of the reduction relation. The de Bruijn development we build on proves the Church–Rosser theorem for $\to_{\mathrm{d}}$. With Theorem 4.13 it becomes a theorem about named terms up to alpha equivalence, with no further work on the named side.

Corollary 4.15 (Church–Rosser for Λ_α). *The relation $\to_\beta$ on Λ_α is confluent: if $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ then there is P with $N_1 \twoheadrightarrow_\beta P$ and $N_2 \twoheadrightarrow_\beta P$.*

Corollary 4.16. *If $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ and both N_1 and N_2 are β-normal, then $N_1 = N_2$.*

4.7. Formalization notes

- *Atoms.* Sections 2 and 3 are parametric in $\mathbb{A}$. The isomorphism is stated for one concrete type of atoms, the natural numbers wrapped in a structure, for which the two hat maps are the projection and the constructor.
- *Permutations.* Norrish and Vestergaard represent a permutation as a list of pairs of strings. The formalization uses finitely supported bijections on atoms. Accordingly inc is not defined by recursion over a list but as the permutation that fixes the atom with number 0 and otherwise acts as π with all numbers shifted by one.
- *Substitution on* dB. CSLib does not define nsub by the equations of Definition 4.1, but as a composition of a lifting, the substitution sub and a lowering of indices. The equations of Definition 4.1 are proved as lemmas.

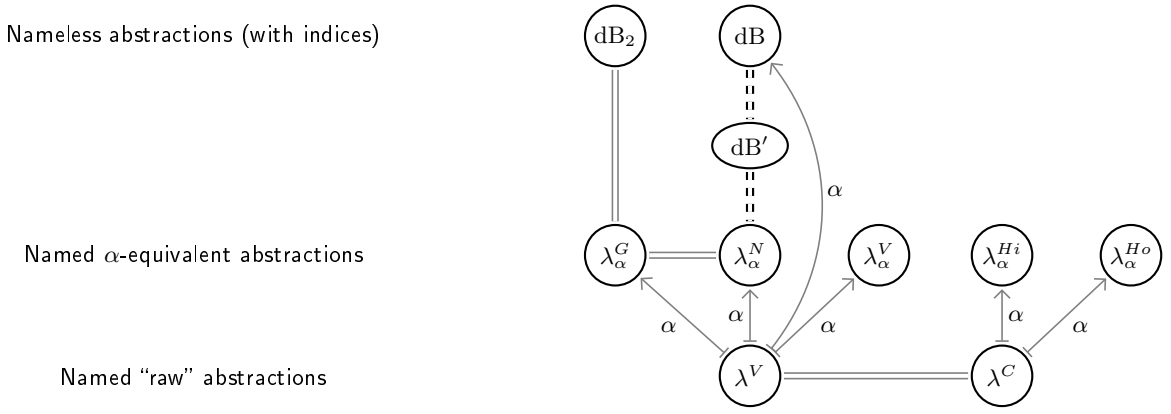


Figure 1: Formalizations of the lambda calculus and their directly proved relationships, redrawn from [12, Figure 1]. Double lines are isomorphisms and arrows are α-quotients. The dashed lines are the results of [12], which Section 4 formalizes.

- *The bijection.* f is a bijection in the sense of an explicit pair of mutually inverse functions. Its inverse is obtained from surjectivity by choice and is not computable.
- *Equivariance.* There is no general notion of an equivariant relation in the library used. In Theorem 4.11 equivariance is a hypothesis on R, and it is proved separately for each of the relations to which the theorem is applied.

5. Related work

Formalizing variable binding has a long history across proof assistants, and several general-purpose approaches exist alongside the direct, elementary style used in CSLib. Nominal techniques [14], as implemented in Nominal Isabelle [16], provide binder-generic reasoning principles via permutations and freshness, at the cost of adopting a dedicated meta-theory. Locally nameless representations [2] aim to make binder-heavy formalizations more tractable. Higher-order abstract syntax sidesteps binder representation issues by delegating them to the proof assistant's own binding mechanism, again at the cost of restricting which proof techniques remain available (e.g. induction principles).

CSLib's current approach is deliberately more elementary: named terms and de Bruijn terms are both represented directly as first-order inductive types, with the relationships between representations (such as the isomorphism of Section 4) proved explicitly rather than assumed via a shared meta-level binding mechanism. This trades some proof automation for representations that are simple to state, and results that transfer cleanly between representations by construction, which fits CSLib's goal of a reusable foundation rather than a self-contained framework.

Mechanized treatments of alpha equivalence as a subject in its own right are fewer. The isomorphism of Section 4 was first mechanized by Norrish and Vestergaard in HOL4 [12]. Crole mentions an Isabelle/HOL development of the results of [5] as work in progress [5, Section 7]; we could find no published account of it. Apinis and Ahman [1] define alpha equivalence of untyped lambda terms as an inductive relation in the Rocq Prover and show that it conforms to the specification given in the literature. Their subject is one definition and its adequacy, whereas Theorem 3.9 compares the definitions that the literature itself uses.

6. Future work

Norrish and Vestergaard place their isomorphism in a web of directly proved relationships between ten formalizations of the lambda calculus, reproduced in Figure 1.

The two results of this paper are intended as the first pieces of a larger, ongoing effort within CSLib to build a formalized foundation for research on untyped lambda calculus. We outline the planned directions below.

6.1. The network of Figure 1

With Section 4, CSLib contains the nodes dB, dB′ and λ^N_α of Figure 1 and the two dashed edges between them. The alpha equivalence by which Λ_α is formed is $\sim_{\mathrm{p}}$, the same relation by which the HOL4 development underlying [12] quotients its raw terms.[6] By Theorem 3.9, Λ_α is equally the quotient by $\sim_{\mathrm{r}}$ and by $\sim_{\mathrm{r}\#}$, the relations of Barendregt [3, Definition 2.1.11] and of Hindley and Seldin [9, Definition 1.17]. Since substitution on Λ_α is lifted from raw terms (Definition 4.3), the construction is also that of λ^{Ho}_α, which Norrish and Vestergaard describe as a quotient of Curry's raw calculus λ^C with substitution lifted to the quotient [12, Section 2]; the raw terms of Section 2.2, with the substitution of Definition 2.4, are such a calculus. Homeier's alpha equivalence, however, is a contextual relation defined without swapping or substitution [10, Definitions 9 and 10], so this edge, too, requires identifying it with one of the five.

Theorem 3.9 thus reduces each remaining α-quotient edge of the figure to identifying the alpha equivalence of a raw calculus with one of the five relations, and the remaining isomorphisms to further representations: Gordon's two-sorted de Bruijn terms dB_2 and Vestergaard's raw calculus λ^V, whose isomorphism with λ^C is mechanized in Coq [12, Section 2.3]. We plan to add these nodes and edges, so that the whole network is formalized within one library.

6.2. Further equivalence results

Crole-style results, showing several natural but superficially different definitions of the same notion to coincide, are not unique to alpha equivalence. We plan analogous formalizations for SK combinator calculus and other binding representations (locally nameless terms, nominal terms), each intended to slot into CSLib alongside the results of this paper and to be connected to them by explicit isomorphisms in the style of Section 4.

6.3. Teaching

Several educators have expressed interest in using the named representation of Section 2.2 to teach the lambda calculus and to base assignments on it. It suits this purpose because it stays close to the textbook presentations: terms with named variables, capture-avoiding substitution, and, through Theorem 3.9, the definitions of alpha equivalence found in the literature [3, 9], all available side by side and proved equivalent. Supporting such use, with documentation and examples aimed at students, is part of our plans for the library.

6.4. Maintenance and reuse

Throughout, the intent is for this work to be maintained as part of CSLib rather than as a standalone, one-off development: subject to code review, kept up to date with CSLib and Mathlib, and structured so that later projects can depend on it directly.

7. Conclusion

We have presented CSLib's current named representation of the untyped lambda calculus, a formalization of Crole's result that five notions of alpha equivalence coincide, and an isomorphism, following Norrish and Vestergaard, between the quotient of named terms under alpha equivalence and a de Bruijn representation. Together these results connect two of the standard approaches to variable binding within a single, reusable Lean 4 library, and form the starting point for a longer-term effort to build a maintained, idiomatic foundation for research on untyped lambda calculus, including further binding representations and the remaining relationships of Figure 1, and to serve in teaching the lambda calculus.

Data availability

The Lean 4 formalization discussed in this paper is developed for CSLib (`https://github.com/leanprover/cslib`). The parts that are not yet merged there are available at `https://github.com/chris-anto-froeschl/cslib`, at the revisions that the links in this paper point to. A blueprint of both formalizations, with a dependency graph showing their connections, is at `https://chris-anto-froeschl.github.io/untyped-lambda-calculus/`.

[6] Rule `aeq_lam` in `examples/lambda/basics/generic_termsScript.sml` of the HOL4 sources.

References

[1] Apinis, K., Ahman, D., 2025. A simple formalization of alpha-equivalence. arXiv:2507.10181.

[2] Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S., 2008. Engineering formal metatheory, in: Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Association for Computing Machinery, New York, NY, USA. p. 3–15. URL: https://doi.org/10.1145/1328438.1328443, doi:10.1145/1328438.1328443.

[3] Barendregt, H.P., 1984. The lambda calculus - its syntax and semantics. volume 103 of *Studies in logic and the foundations of mathematics*. Revised ed., North-Holland.

[4] Barrett, C., Chaudhuri, S., Montesi, F., Grundy, J., Kohli, P., de Moura, L., Rademaker, A., Yingchareonthawornchai, S., 2026. Cslib: The lean computer science library. URL: https://arxiv.org/abs/2602.04846, arXiv:2602.04846.

[5] Crole, R.L., 2012. Alpha equivalence equalities. Theoretical Computer Science 433, 1–19. URL: https://www.sciencedirect.com/science/article/pii/S0304397512000667, doi:10.1016/j.tcs.2012.01.030.

[6] Gabbay, M.J., 2011. Foundations of nominal techniques: logic and semantics of variables in abstract syntax. Bulletin of Symbolic Logic 17, 161–229.

[7] Gabbay, M.J., Mathijssen, A., 2008. Capture-avoiding substitution as a nominal algebra. Formal Aspects of Computing 20, 451–479.

[8] Gabbay, M.J., Pitts, A.M., 2002. A new approach to abstract syntax with variable binding. Formal Aspects of Computing 13, 341–363. doi:10.1007/s001650200016.

[9] Hindley, J.R., Seldin, J.P., 2008. Lambda-Calculus and Combinators: An Introduction. 2 ed., Cambridge University Press.

[10] Homeier, P.V., 2001. A proof of the Church-Rosser theorem for the lambda calculus in higher order logic, in: Boulton, R.J., Jackson, P.B. (Eds.), TPHOLs 2001: Supplemental Proceedings, Division of Informatics, University of Edinburgh. pp. 207–222. Informatics Research Report EDI-INF-RR-0046.

[11] Nipkow, T., 2001. More Church-Rosser proofs (in Isabelle/HOL). Journal of Automated Reasoning 26, 51–66.

[12] Norrish, M., Vestergaard, R., 2007. Proof pearl: De bruijn terms really do work, in: Schneider, K., Brandt, J. (Eds.), Theorem Proving in Higher Order Logics, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 207–222.

[13] Pitts, A.M., 2006. Alpha-structural recursion and induction. Journal of the ACM 53, 459–506.

[14] Pitts, A.M., 2013. Nominal Sets: Names and Symmetry in Computer Science. volume 57 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press.

[15] Urban, C., Pitts, A.M., Gabbay, M.J., 2004. Nominal unification. Theoretical Computer Science 323, 473–497.

[16] Urban, C., Tasson, C., 2005. Nominal techniques in isabelle/hol, in: Nieuwenhuis, R. (Ed.), Automated Deduction – CADE-20, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 38–53.